Smart ICU Monitoring & Alert System Project Report

Chevuru V R Dinesh Karthik (BT2024199)

---

1. Problem Statement

The objective of this project is to design and implement a distributed, real-time Smart Intensive Care Unit (ICU) Monitoring System. It allows concurrent clients—nurses, doctors, and admins—to securely connect to a centralized server. The server manages active sessions, simulates the continuous streaming of patient vitals, detects life-threatening medical anomalies via moving averages and static thresholds, and efficiently dispatches real-time alerts to the appropriate medical staff.

The system demonstrates key operating system concepts such as process management, synchronization, inter-process communication, file locking, socket programming, and role-based access control.

2. Implementation of Required Concepts

2.1 Socket Programming

The system uses TCP socket programming to enable reliable, continuous communication between multiple client processes (Admin, Doctor, Nurse, Guest) and the central server.

Functions used:

- socket(), bind(), listen(), accept() (Server)
- connect(), send(), recv() (Clients)

2.2 Threading and Concurrency Control

The server uses a thread-per-client model using POSIX threads to handle multiple active sessions simultaneously, allowing dozens of nurses to continuously stream vitals concurrently without blocking the server.

Functions used:

- pthread_create()
- pthread_detach()

2.3 Synchronization and Data Consistency

Mutex locks are used to ensure safe read/write access to the shared patient database and user tables in memory, preventing race conditions, dirty reads, and lost updates. Semaphores are used to limit the number of concurrent disk I/O operations to prevent disk thrashing.

Functions used:

- pthread_mutex_lock(), pthread_mutex_unlock()
- sem_init(), sem_wait(), sem_post()

2.4 Inter-Process Communication (IPC)

The system uses three IPC mechanisms to route emergency medical alerts:

1. Signals: kill (pid, SIGUSR1) interrupts the doctor's client process to notify them of a critical alert.
2. Named Pipes (FIFOs): /tmp/icu_alert_<id> passes the textual details of anomalies asynchronously.
3. Message Queues: POSIX mq_open (/icu_mqueue) acts as a system-wide bulletin board for high-severity events.

2.5 Authentication and Role-Based Access

The system enforces role-based access on the server-side:

- Admin: Full access to manage users, patients, and assignments. Cannot self-delete.
- Doctor: Can only view medical histories of assigned patients and receive real-time alerts.
- Nurse: Can only access and stream live vitals for assigned patients.
- Guest: Read-only access to anonymized ICU statistics.

3. Screenshots / Output

3.1 Server Startup and Daemon

```
dkarthik38@pop-os:~/Desktop/OSminiProject$ ./bin/icu_server

╔═══════════════════════════════════════════════════════════╗
║           SMART ICU MONITORING & ALERT SYSTEM v1.0        ╕
╚═══════════════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════════════╗
║                Server — OS Mini Project                   ╕
╚═══════════════════════════════════════════════════════════╝

[2026-04-30 22:36:51] === Server starting ===
[2026-04-30 22:36:51] Loaded 6 users from data/users.dat
[2026-04-30 22:36:51] Loaded 10 patients from disk
[2026-04-30 22:36:51] POSIX message queue opened: /icu_mqueue
[2026-04-30 22:36:51] Named pipe ready: /tmp/icu_alert_2
[2026-04-30 22:36:51] Named pipe ready: /tmp/icu_alert_3

[SERVER] Listening on port 8888  (Ctrl+C to stop)

[2026-04-30 22:36:51] Listening on port 8888
```

3.2 Role-Based Login and Admin Menu

```
dkarthik38@pop-os:~/Desktop/OSminiProject$ ./bin/icu_client

========================================
         SMART ICU MONITORING
========================================
Select your role to login:
  [1] Admin
  [2] Doctor
  [3] Nurse
  [4] Guest
  [0] Exit
Choice: 1
╔═══════════════════════════════════════════════════════════╗
║                       ICU ADMIN CLIENT                    ╕
╚═══════════════════════════════════════════════════════════╝
Connecting to 127.0.0.1:8888 ...
Login attempt 1/3
Username: admin
Password: admin123
✓ Authenticated as Admin


╔═ ADMIN MENU ═══════════════════════════╗
║ [1] List Users        [2] Add User     ╕
║ [3] Delete User       [4] List Patients╕
║ [5] Add Patient       [6] Discharge Pt ╕
║ [7] Assign Nurse      [8] Server Stats ╕
║ [9] Quit                               ╕
╚═══════════════════════════════════════╝
Choice:
```

3.3 Nurse Streaming Vitals

```
╔═══════════════════════════════════════════════════════════╗
║                        ICU NURSE CLIENT                   ║
╚═══════════════════════════════════════════════════════════╝
Connecting to 127.0.0.1:8888 …
ICU_SERVER 1.0 — authenticate with: AUTH <user> <pass>
Login attempt 1/3
Username: nurse1
Password: nurse123
✓ Authenticated as Nurse (id=4)
READY nurse nurse1

Assigned patients:
  [1] Patient #109 — dk3
  [2] Patient #108 — dk2
  [3] Patient #110 — dk4
  [4] Patient #107 — dk
Select patient [1-4]: 1

[NURSE] Monitoring patient #109 — dk3

──────────────────────────────────────────────────────────
Streaming vitals every 3 s.
  Press Enter at any time to pause & open the menu.

  [2026-04-30 22:40:02]  HR: 84.5  BP: 103/77  SpO2: 97.8%  Temp: 36.4°C  RR: 17
  Recorded.
  [2026-04-30 22:40:05]  HR: 83.7  BP: 109/69  SpO2: 99.4%  Temp: 36.4°C  RR: 6
  *** SERVER ALERT: CRITICAL Resp Rate=6.0(CRITICAL)
  [2026-04-30 22:40:08]  HR: 81.4  BP: 115/72  SpO2: 99.2%  Temp: 36.9°C  RR: 14
  Recorded.
  [2026-04-30 22:40:11]  HR: 82.5  BP: 130/72  SpO2: 98.4%  Temp: 39.4°C  RR: 14
  *** SERVER ALERT: CRITICAL Temperature=39.4(CRITICAL)  Temp σ-outlier(39.4 vs avg 37.0)
  [2026-04-30 22:40:14]  HR: 74.7  BP: 128/77  SpO2: 96.3%  Temp: 36.6°C  RR: 19

──────────────────────────────────────────────────────────
  ▪  STREAMING PAUSED
──────────────────────────────────────────────────────────
  Current patient: #109 — dk3
```

3.4 Doctor Receiving IPC Alerts

```
╔═══════════════════════════════════════════════════════════╗
║                        ICU DOCTOR CLIENT                  ║
╚═══════════════════════════════════════════════════════════╝
Connecting to 127.0.0.1:8888 ...
Login attempt 1/3
Username: drsmith
Password: doc123
✓ Authenticated as Doctor (id=2)

✓ PID 14886 registered for real-time alerts.
✓ Alert pipe: /tmp/icu_alert_2


  Alerts from the server will appear here automatically
  as they arrive — no need to wait or poll.

╔═ DOCTOR MENU ═══════════════════════════╗
║ [1] My Patients                          ║
║ [2] View Patient Details & Vitals        ║
║ [3] Quit                                 ║
╚═══════════════════════════════════════════╝
Choice:
┌─ ALERT ──────────────────────────────────────────────┐
│ [CRITICAL] 2026-04-30 22:40:08
│ Patient #109 — dk3
│ Resp Rate=6.0(CRITICAL)
└──────────────────────────────────────────────────────┘


┌─ ALERT ──────────────────────────────────────────────┐
│ [CRITICAL] 2026-04-30 22:40:14
│ Patient #109 — dk3
│ Temperature=39.4(CRITICAL)  Temp σ-outlier(39.4 vs avg 37.0)
└──────────────────────────────────────────────────────┘
```

3.5 System Event Logging

```
logs > server.log
[2026-04-30 22:36:51] === Server starting ===
[2026-04-30 22:36:51] Loaded 6 users from data/users.dat
[2026-04-30 22:36:51] Loaded 10 patients from disk
[2026-04-30 22:36:51] POSIX message queue opened: /icu_mqueue
[2026-04-30 22:36:51] Named pipe ready: /tmp/icu_alert_2
[2026-04-30 22:36:51] Named pipe ready: /tmp/icu_alert_3
[2026-04-30 22:36:51] Listening on port 8888
[2026-04-30 22:37:32] Connection from 127.0.0.1:41096
[2026-04-30 22:37:36] Login: admin [ADMIN]
[2026-04-30 22:38:27] Connection from 127.0.0.1:42020
[2026-04-30 22:38:31] Login: nurse1 [NURSE]
[2026-04-30 22:38:40] ALERT [CRITICAL] Patient#109 -> Doctor#2 : SpO2=100.0(CRITICAL)
[2026-04-30 22:38:40] Named pipe /tmp/icu_alert_2 not open (doctor offline?)
[2026-04-30 22:38:40] Alert enqueued in mqueue
[2026-04-30 22:38:40] [ANOMALY] Patient#109 CRITICAL: SpO2=100.0(CRITICAL)
[2026-04-30 22:39:36] Connection from 127.0.0.1:35580
[2026-04-30 22:39:44] Login: drsmith [DOCTOR]
[2026-04-30 22:39:44] Doctor drsmith registered pid=14886
[2026-04-30 22:39:54] Connection from 127.0.0.1:57100
[2026-04-30 22:39:58] Login: nurse1 [NURSE]
[2026-04-30 22:40:08] ALERT [CRITICAL] Patient#109 -> Doctor#2 : Resp Rate=6.0(CRITICAL)
[2026-04-30 22:40:08] Alert written to named pipe /tmp/icu_alert_2
[2026-04-30 22:40:08] SIGUSR1 sent to doctor pid=14886
[2026-04-30 22:40:08] Alert enqueued in mqueue
[2026-04-30 22:40:08] [ANOMALY] Patient#109 CRITICAL: Resp Rate=6.0(CRITICAL)
[2026-04-30 22:40:14] ALERT [CRITICAL] Patient#109 -> Doctor#2 : Temperature=39.4(CRITICAL)  Temp σ-outlier(39.4 vs avg 37.0)
[2026-04-30 22:40:14] Alert written to named pipe /tmp/icu_alert_2
[2026-04-30 22:40:14] SIGUSR1 sent to doctor pid=14886
[2026-04-30 22:40:14] Alert enqueued in mqueue
[2026-04-30 22:40:14] [ANOMALY] Patient#109 CRITICAL: Temperature=39.4(CRITICAL)  Temp σ-outlier(39.4 vs avg 37.0)
[2026-04-30 23:20:49] === Server starting ===
[2026-04-30 23:20:49] Loaded 6 users from data/users.dat
[2026-04-30 23:20:49] Loaded 10 patients from disk
[2026-04-30 23:20:49] POSIX message queue opened: /icu_mqueue
[2026-04-30 23:20:49] Named pipe ready: /tmp/icu_alert_2
[2026-04-30 23:20:49] Named pipe ready: /tmp/icu_alert_3
[2026-04-30 23:20:49] Listening on port 8888
```

4.Challenges Faced and Solutions

4.1 Terminal Input/Output Interference

Challenge:
The Nurse client needed to accept an "Enter" keypress to pause streaming while printing vitals every 3 seconds. Using scanf() caused blocking and froze streaming.

Solution:
A background input-watcher thread was implemented using non-blocking raw terminal mode (termios) and a shared volatile pause flag protected by a mutex. This separated streaming from input handling.

4.2 Dynamic State Synchronization

Challenge:
When a Nurse paused their stream to access the "Switch Patient" menu and an

Admin concurrently assigned them a new patient, the Nurse menu did not reflect the updated assignment because the state was cached locally.

Solution:
The client-side menu loop was restructured to dynamically perform a TCP LIST_PATIENTS request within each refresh cycle. This ensures the client always reflects the latest server state without requiring a re-login.

4.3 Infinite Loop on Disconnection (EOF)

Challenge:
If a client abruptly closed its input stream (for example, using Ctrl+D or due to unexpected server termination), the CLI menus entered an infinite loop because scanf failed without consuming the EOF character.

Solution:
EOF detection was implemented using feof(stdin), along with input buffer draining using a loop (while((c = getchar()) != '\n' && c != EOF)). This allowed the system to detect disconnections and exit gracefully.

4.4 Data Persistence and File Corruption Risks

Challenge:
The system needed to persist patient vitals and user data into binary .dat files. With multiple clients modifying data simultaneously, there was a high risk of race conditions leading to corrupted or partially written files.

Solution:
POSIX advisory file locking was implemented using fcntl(). Exclusive write locks (F_WRLCK) were applied before writing, and read locks (F_RDLCK) before reading. Additionally, a POSIX semaphore (sem_t) was used to limit concurrent disk I/O operations and prevent disk thrashing.

4.5 Asynchronous Alert Delivery

Challenge:
Doctors needed to receive critical patient alerts instantly, even while navigating menus or viewing other data. Traditional polling mechanisms were insufficient for real-time responsiveness.

Solution:
A multi-layered IPC strategy was implemented. POSIX signals (kill(pid, SIGUSR1)) were used to interrupt the doctor's process immediately. A background thread in the doctor's client then reads detailed alert information from a Named Pipe (FIFO) and displays it seamlessly.

5. Conclusion

The Smart ICU Monitoring System successfully achieves its goal of simulating a highly concurrent, real-time distributed application. By integrating core operating system concepts such as multi-threading, mutex synchronization, POSIX inter-process communication (signals, FIFOs, and message queues), and file locking, the project demonstrates a strong practical understanding of managing shared resources and coordinating complex processes.

The system remains stable under concurrent workloads and ensures reliable, real-time delivery of critical medical alerts without data loss, making it both efficient and robust.